



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



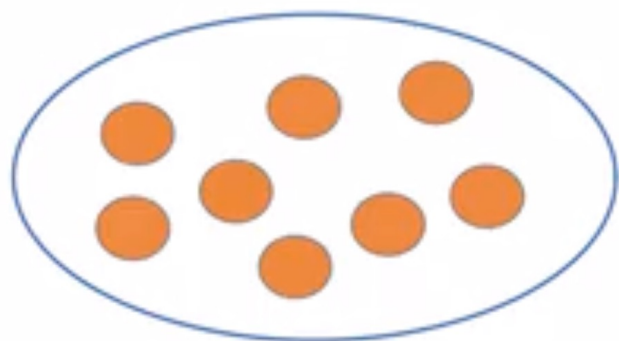
第六章 集合与字典



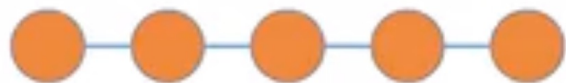
- 集合及其表示
- 并查集与等价类
- 字典
- 散列



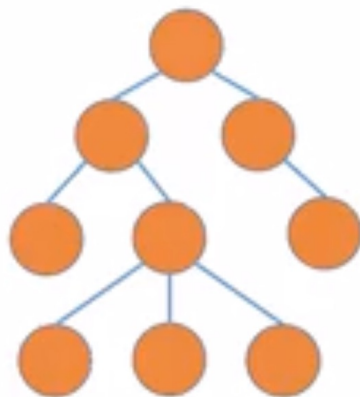
逻辑结构——数据元素之间的逻辑关系是什么？



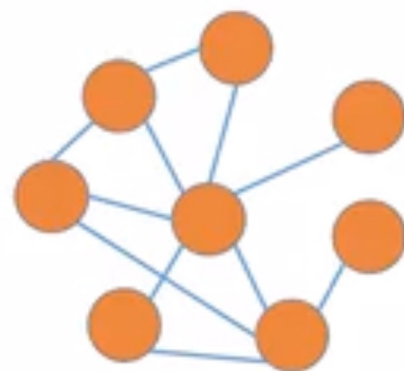
集合



线性结构



树形结构



图结构



6.1 集合及其表示



6.1.1 集合基本概念

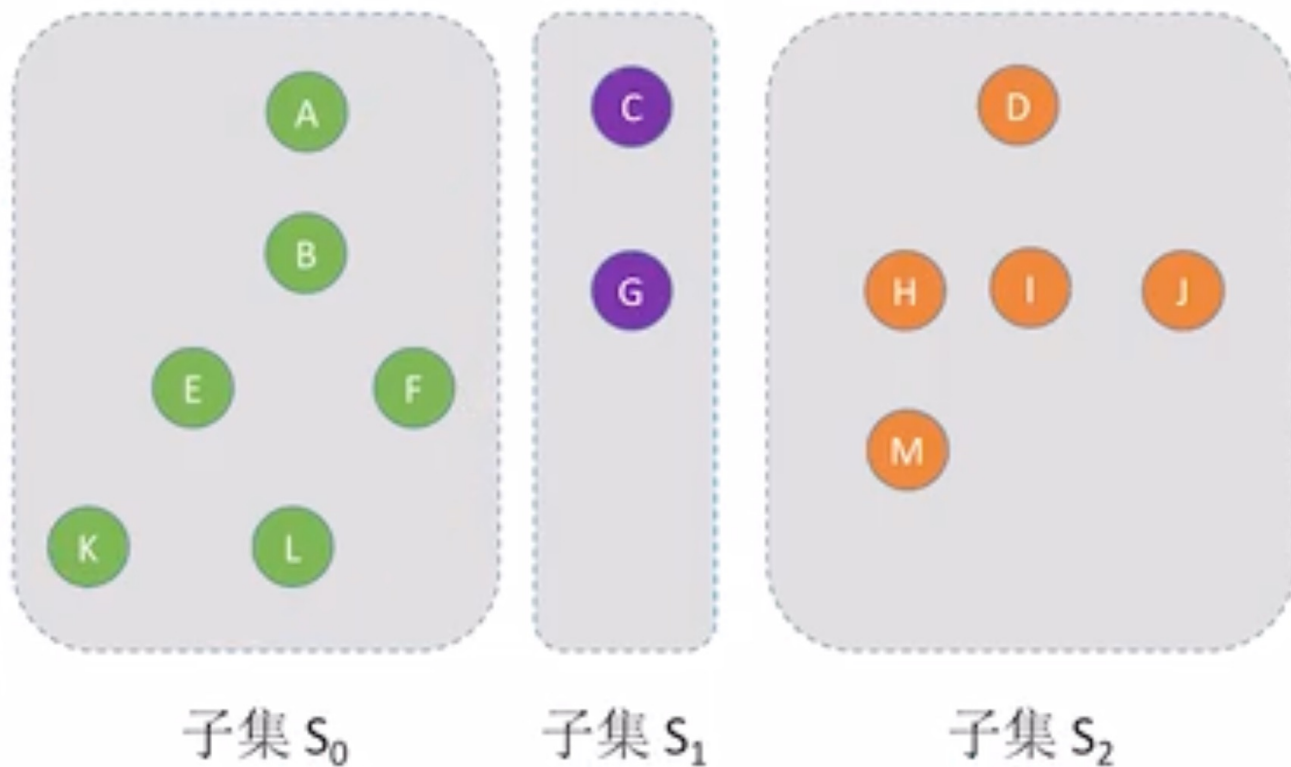
- 集合是**成员**(元素)的一个群集。集合中的成员可以是**原子**(**单元元素**)，也可以是**集合**。
- 集合的成员必须互不相同。
- 在算法与数据结构中所遇到的集合，其单个元素通常是整数、字符、字符串或指针，且同一集合中所有成员具有**相同的数据类型**。
- 例： $\text{colour} = \{ \text{red, orange, yellow, green, black, blue, purple, white} \}$



- 集合中的成员**一般**是无序的，但在表示它时，常写在一个序列里。
- 常设定集合中的单个元素具有线性有序关系，此关系可记作“ $<$ ”，表示“优先于”。
- 整数、字符和字符串都有一个自然线性顺序。指针也可依据其在序列中安排的位置给予一个线性顺序。

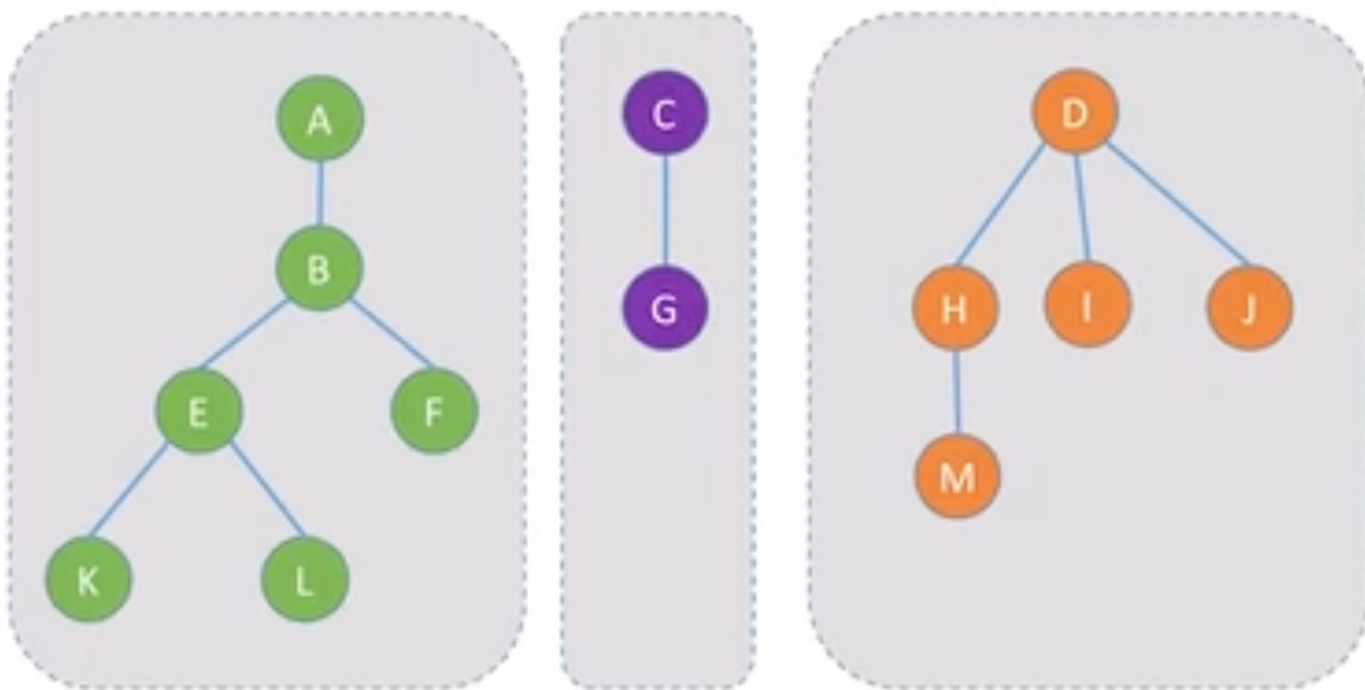


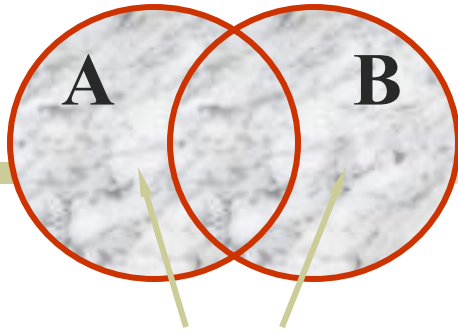
将各个元素划分为若干个互不相交的子集



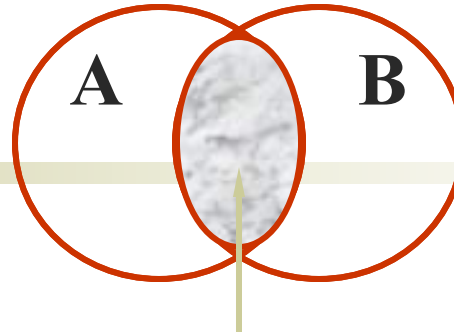


用互不相交的树，表示多个“集合”

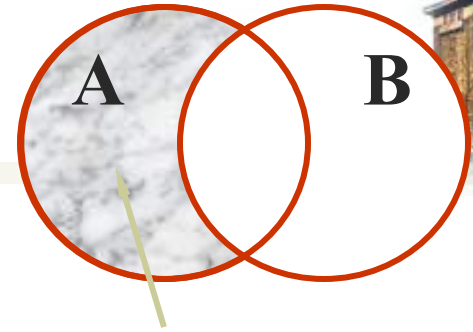




$A \cup B$ 或 $A + B$



$A \cap B$ 或 $A \times B$



$A - B$

集合 (Set) 的抽象数据类型

```
template <class T>
```

```
class Set {
```

```
public:
```

```
    virtual Set() = 0;           //构造函数
```

```
    virtual makeEmpty() = 0;     //置空集合
```

```
    virtual bool addMember (const T x) = 0;
```




```
virtual bool delMember (const T x) = 0;  
virtual Set<T> intersectWith (Set<T>& R) = 0;  
    //集合的交运算  
virtual Set<T> unionWith (Set<T>& R) = 0;  
    //集合的并运算  
virtual Set<T> differenceFrom (Set<T>& R) = 0;  
    //集合的差运算  
virtual bool Contains (T x) = 0;  
virtual bool subSet (Set<T>& R) = 0;  
virtual bool operator == (Set<T>& R) = 0;  
    //判集合是否相等
```

```
};
```



6.1.2 用位向量实现集合抽象数据类型



- 当集合是全集合 $\{0, 1, 2, \dots, n\}$ 的一个子集, 且 n 是不大的整数时, 可用位(0, 1)向量来实现集合。
- 当全集合是由有限个可枚举的成员组成时, 可建立全集合成员与整数 $0, 1, 2, \dots$ 的一一对应关系, 用位向量来表示该集合的子集。



- 一个二进位有两个取值1或0，分别表示在集合与不在集合。如果采用16位无符号短整数数组 `bitVector[ ]` 作为集合的存储，就要考虑如何求出元素 `i` 在 `bitVector` 数组中的相应位置。
- 这种实现方式非常高效。
- `getMember` 和 `putMember` 的时间复杂度都是 $O(1)$ ，并且对于稠密且范围确定的整数集合，它比基于链表或哈希表的实现更加节省内存。



集合的位向量(bit Vector)类的定义



```
#include <assert.h>
#include <iostream.h>
const int DefaultSize = 50;
template <class T>
class bitSet {
//用位向量来存储集合元素, 集合元素的范围在0到
//setSize-1之间。数组采用16位无符号短整数实现
public:
    bitSet (int sz = DefaultSize);           //构造函数
    bitSet (const bitSet<T>& R);             //复制构造函数
    ~bitSet () { delete [ ]bitVector; }     //析构函数
    unsigned short getMember (const T x);    //读取集合元素x
```



```
void putMember (const T x, unsigned short v);  
                                     //将值v送入集合元素x  
void makeEmpty () {  
    for (int i = 0; i < vectorSize; i++)  
        bitVector[i] = 0;  
}  
bool addMember (const T x);    //加入新成员x  
bool delMember (const T x);    //删除老成员x  
bitSet<T>& operator = (const bitSet<T>& R);  
bitSet<T>& operator + (const bitSet<T>& R);  
bitSet<T>& operator * (const bitSet<T>& R);  
bitSet<T>& operator - (const bitSet<T>& R);  
bool Contains (const T x);    //判x是否集合this的成员
```



bool subSet (bitSet<T>& R); //判this是否R的子集

bool operator == (bitSet<T>& R);

//判集合this与R相等

**friend istream& operator >> (istream& in,
bitSet<T>& R);**

//输入

**friend ostream& operator << (ostream& out,
bitSet<T>& R);**

//输出

private:

int setSize; //集合大小

int vectorSize; //位数组大小

unsigned short *bitVector; //存储集合元素的位数组

};



构造函数的实现



```
template <class T>
bitSet<T>::bitSet (int sz) : setSize (sz) { //构造函数
    assert (setSize > 0);                //检查参数的合理性
    vectorSize = (setSize+15) >> 4;      //存储数组大小
    bitVector = new unsigned short[vectorSize];
                                           //分配空间
    assert (bitVector != NULL);
                                           //检查存储分配是否成功
    for (int i = 0; i < vectorSize; i++)
        bitVector[i] = 0;                //初始化
};
```



```
template<class T>
```

```
unsigned short bitSet<T>:: getMember (const T x) {
```

```
    //读取集合元素x, x从0开始
```

```
    int ad = x/16;
```

```
    int id = x%16;
```

```
    unsigned short elem = bitVector[ad];
```

```
    return ((elem >> (15 - id)) %2);
```

```
};
```




```
template<class T>
```

```
void bitSet<T>::putMember (const T x, unsigned  
    short v) { //将值v送入集合元素x  
    int ad = x/16;  
    int id = x%16;  
    unsigned short elem = bitVector[ad];  
    unsigned short temp = elem >> (15-id);  
    elem = elem << (id+1);  
    if (temp%2==0 && v ==1) temp = temp +1;  
    else if (temp%2==1 && v==0) temp = temp -1;  
    bitVector[ad] = (temp<<(15-id)) | (elem >> (id+1));  
};
```



```
template<class T>
```

```
bitSet<T>& bitSet<T>::      //求集合this与R的并
```

```
operator + (const bitSet<T>& R) {
```

```
    assert (vectorSize == R.vectorSize);
```

```
    bitSet temp (vectorSize);
```

```
    for (int i = 0; i < vectorSize; i++)
```

```
        temp.bitVector[i] = bitVector[i] | R.bitVector[i];
```

```
    return temp; //按位求或, 由第三个集合返回  
};
```



```
template <class T>
```

```
bitSet<T>& bitSet<T>::          //求集合this与R的交
```

```
operator * (const bitSet<T>& R) {
```

```
    assert (vectorSize == R.vectorSize);
```

```
    bitSet temp (setSize);
```

```
    for (int i = 0; i < vectorSize; i++)
```

```
        temp.bitVector[i] = bitVector[i] & R.bitVector[i];
```

```
    return temp; //按位求 “与” , 由temp返回
```

```
};
```



```
template <class T>
```

```
bitSet<T>& bitSet<T>::          //求集合this与R的差
```

```
operator - (const bitSet<T>& R) {
```

```
    assert (vectorSize == R.vectorSize);
```

```
    bitSet temp (setSize);
```

```
    for (int i = 0; i < vectorSize; i++)
```

```
        temp.bitVector[i] = bitVector[i] & (~R.bitVector[i]);
```

```
    return temp;                //由第三个集合返回
```

```
};
```



集合的并

this

0	1	1	1	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---

R

0	0	1	0	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---

temp

0	1	1	1	0	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---

集合的交

this

0	1	1	1	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---

R

0	0	1	0	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---

temp

0	0	1	0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---

集合的差

this

0	1	1	1	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---

R

0	0	1	0	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---

temp

0	1	0	1	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---



```
template <class T>
```

```
bool bitSet<T>::subSet (bitSet<T>& R) {
```

```
//判this是否R的子集
```

```
    assert (setSize == R.setSize);
```

```
    for (int i = 0; i < vectorSize; i++) //按位判断
```

```
        if (bitVector[i] & (~R.bitVector[i])) return false;
```

```
    return true;
```

```
};
```

this

0	0	1	1	1	0	1	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---

R

0	0	1	1	1	0	0	1	0	1	0
1	1	0	0	0	1	1	0	1	0	1

×



```
template <class T>
```

```
bool bitSet<T>::operator == (bitSet<T>& R) {
```

```
//判集合this与R相等
```

```
    if (vectorSize != R.vectorSize) return false;
```

```
    for (int i = 0; i < vectorSize; i++)
```

```
        if (bitVector[i] != R.bitVector[i])
```

```
            return false;
```

```
    return true;
```

```
//对应位全部相等
```

```
};
```

this

0	0	1	1	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---

R

0	0	1	0	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---

↑
i



使用示例



```
bitSet<int> s1, s2, s3, s4, s5; bool index, equal;  
for (int k = 0; k < 10; k++) {           //集合赋值  
    s1.addMember(k);    //s1: {0, 1, 2, ..., 9}  
    s2.addMember(k+7);  //s2: {7, 8, 9, ..., 16}  
}  
  
s3 = s1+s2;    //求s1与s2的并 {0, 1, ..., 16}  
s4 = s1*s2;    //求s1与s2的交 {7, 8, 9}  
s5 = s1-s2;    //求s1与s2的差 {0, 1, ..., 6}
```




//s1: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

//s4: {7, 8, 9}

index = s1.subSet (s4); //判断s1是否为s4子集

cout << index << endl; //结果, **index = 0**

equal = (s1 == s2); //比较集合s1与s2是否相等

cout << equal << endl; //为0, 两集合不等



主要优点



1. 极高的空间效率

- 核心优势：每个元素只占用 **1个比特（bit）** 的存储空间。
- 对比：如果使用链表或数组，每个元素至少需要存储数据本身（如4字节的int）和指针（4或8字节），而位向量只需要1位。

2. 常数时间复杂度的基本操作

- 查找（**isMember**）： $O(1)$ - 直接计算位位置并检查
- 插入（**add**）： $O(1)$ - 直接设置对应位为1
- 删除（**remove**）： $O(1)$ - 直接设置对应位为0



3. 高效的集合运算

- 位向量支持通过位级并行操作一次性处理整个字（通常32或64位），使得集合运算极其高效：
- 并集 ($\mathbf{A \cup B}$) : `bit_vector_A | bit_vector_B`
- 交集 ($\mathbf{A \cap B}$) : `bit_vector_A & bit_vector_B`
- 差集 ($\mathbf{A - B}$) : `bit_vector_A & ~bit_vector_B`
- 对称差集: `bit_vector_A ^ bit_vector_B`

4. 缓存友好性

- 数据在内存中连续存储，具有很好的空间局部性
- CPU缓存可以高效地预取和处理连续的内存块
- 相比链表的随机内存访问，性能更好

5. 硬件加速

- 现代CPU的位操作指令（AND, OR, XOR, NOT）都是单周期或接近单周期指令
- 可以充分利用处理器的位操作并行性



限制和缺点



- **1. 元素范围必须预先确定**
 - 需要知道最大可能的元素值
 - 动态扩展比较困难
- **2. 稀疏集合浪费空间**
 - 如果集合很稀疏（只有很少元素），比如：全集大小：1,000,000；实际元素：10个，仍然需要125KB空间；
 - 链表只需要约120字节。
- **3. 仅限于整数类型**
 - 元素必须是整数或可映射到整数
 - 不支持复杂的对象或字符串作为元素
- **4. 不支持元素排序**
 - 位向量本身不保持元素的任何顺序信息
 - 如果需要有序遍历，需要额外处理

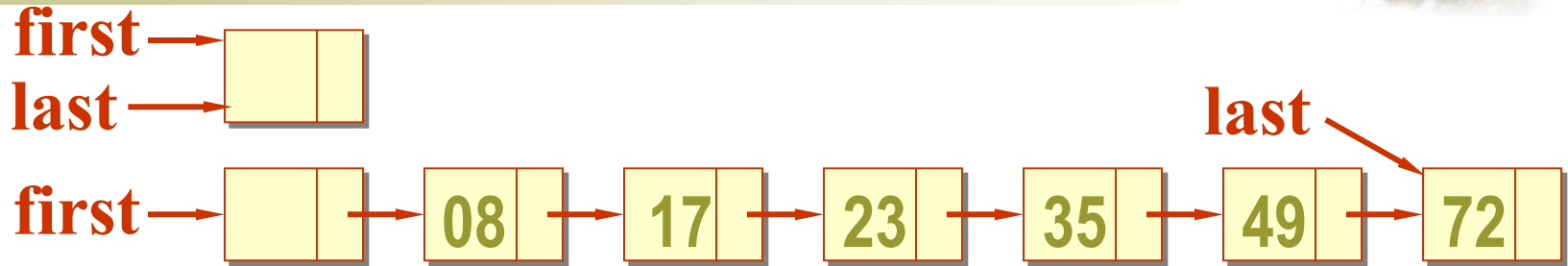


- 使用位向量实现集合的黄金法则：
 - 当元素是密集的整数且需要频繁的集合运算时，位向量是无可争议的最佳选择。
- 对于元素稀疏或需要存储复杂对象的情况，则应考虑哈希表、平衡树或链表等其他数据结构。

操作	位向量	有序链表	哈希表
查找	$O(1)$	$O(n)$	$O(1)$
插入	$O(1)$	$O(n)$	$O(1)$
删除	$O(1)$	$O(n)$	$O(1)$
并集	$O(n/w)$	$O(m+n)$	$O(m+n)$
交集	$O(n/w)$	$O(m+n)$	$O(\min(m,n))$
空间	最优	较差	中等



6.1.3 用有序链表实现集合抽象数据类型



用带表头结点的有序链表表示集合

- 用有序链表来表示集合时，链表中的每个结点表示集合的一个成员。
- 各结点所表示的成员 $e_0, e_1, \dots, e_n$ 在链表中**按升序排列**，即 $e_0 < e_1 < \dots < e_n$ 。
- 集合成员可以无限增加。因此，用有序链表可以表示无穷全集集合的子集。



集合的有序链表类的定义



```
template <class T>
struct SetNode {           //集合的结点类定义
    T data;                //每个成员的数据
    SetNode<T> *link;      //链接指针
    SetNode() : link (NULL); //构造函数
    SetNode (const T& x, SetNode<T> *next = NULL)
        : data (x), link (next); //构造函数
};
```



```
template <class T>
```

```
class LinkedSet { //集合的类定义
```

```
private:
```

```
    SetNode<T> *first, *last;
```

```
    //有序链表表头指针, 表尾指针
```

```
public:
```

```
    LinkedSet () { first = last = new SetNode<T>; }
```

```
    LinkedSet (LinkedSet<T>& R); //复制构造函数
```

```
    ~LinkedSet () { makeEmpty(); delete first; }
```

```
    void makeEmpty(); //置空集合
```

```
    bool addMember (const T& x);
```

```
    bool delMember (const T& x);
```

```
    LinkedSet<T>& operator = (LinkedSet<T>& R);
```

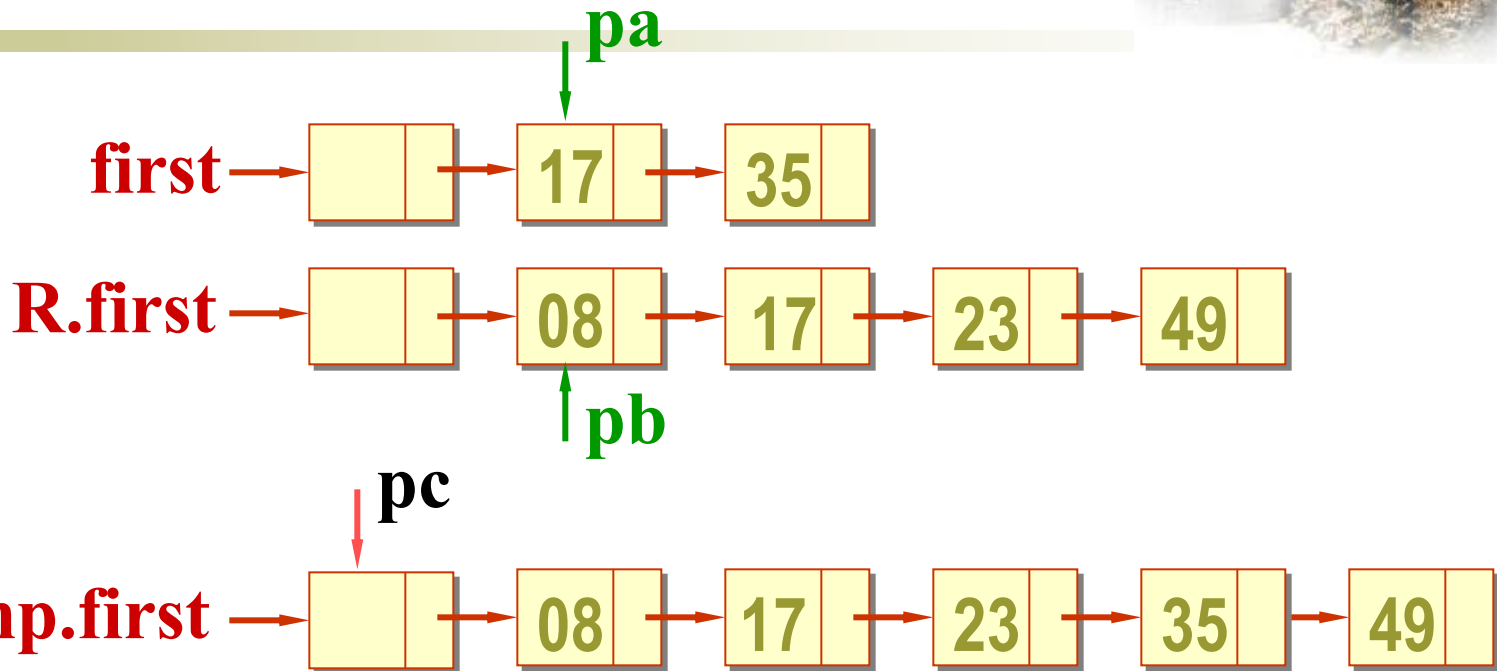
```
    LinkedSet<T>& operator + (LinkedSet<T>& R);
```




```
LinkedSet<T>& operator * (LinkedSet<T>& R);  
LinkedSet<T>& operator - (LinkedSet<T>& R);  
bool Contains (const T x); //判x是否集合的成员  
bool operator == (LinkedSet<T>& R);  
                        //判集合this与R相等  
bool Min (T& x);      //返回集合最小元素的值  
bool Max (T& x);      //返回集合最大元素的值  
bool subSet (LinkedSet<T >& R);  
                        //判this是否R的子集  
};
```



表示集合的几个重载函数



```
template <class T>
```

```
LinkedSet<T>& LinkedSet<T>::
```

```
operator + (LinkedSet<T>& R) {
```

```
    //求集合this与集合R的并
```



```
SetNode<T> *pb = R.first->link; //R扫描指针
SetNode<T> *pa = first->link;   //this扫描指针
LinkedSet<T> temp;               //创建空链表
SetNode<T> *p, *pc = temp.first; //结果存放指针
while (pa != NULL && pb != NULL) {
    if (pa->data == pb->data) {    //两集合共有
        pc->link = new SetNode<T>(pa->data);
        pa = pa->link; pb = pb->link;
    }
    else if (pa->data < pb->data) { //this元素值小
        pc->link = new SetNode<T>(pa->data);
        pa = pa->link;
    }
}
```



```
else {                                     //R集合元素值小
    pc->link = new SetNode<T>(pb->data);
    pb = pb->link;
}
pc = pc->link;
}
if (pa != NULL) p = pa;    //this集合未扫完
else p = pb;               //或R集合未扫完
while (p != NULL) {        //向结果链逐个复制
    pc->link = new SetNode<T>(p->data);
    pc = pc->link; p = p->link;
}
pc->link = NULL; temp.last = pc;    //链表收尾
return temp;
};
```



- 1. 使用有序链表实现集合的主要优点是:
 - A) 内存使用效率最高
 - B) 所有操作时间复杂度都是 $O(1)$
 - C) 便于执行集合的并、交、差等运算
 - D) 插入和删除操作最快
- 2. 在有序链表集合中查找元素时，平均时间复杂度是:
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n^2)$
- 3. 向有序链表集合中插入新元素时，首先需要:
 - A) 直接插入到链表头部
 - B) 找到合适的插入位置以保持有序性
 - C) 随机选择一个位置插入
 - D) 先排序整个链表再插入



- 给定有序链表集合 $A = 1 \rightarrow 3 \rightarrow 5 \rightarrow 7$, $B = 2 \rightarrow 3 \rightarrow 6 \rightarrow 7$, 求:
 - $A \cup B =$ _____
 - $A \cap B =$ _____
 - $A - B =$ _____
- 判断正误: 有序链表集合适合频繁插入删除但很少查询的场景。



6.2 并查集与等价类



6.2.1 并查集 (Union-Find Sets)

- 并查集支持以下三种操作：
 - ◆ Union (Root1, Root2) //合并操作
 - ◆ Find (x) //查找操作
 - ◆ UFSets (s) //构造函数
- 对于并查集来说，每个集合用一棵树表示。
- 为此，采用树的双亲表示作为集合存储表示。集合元素的编号从0到 $n-1$ ，其中 n 是最大元素个数。

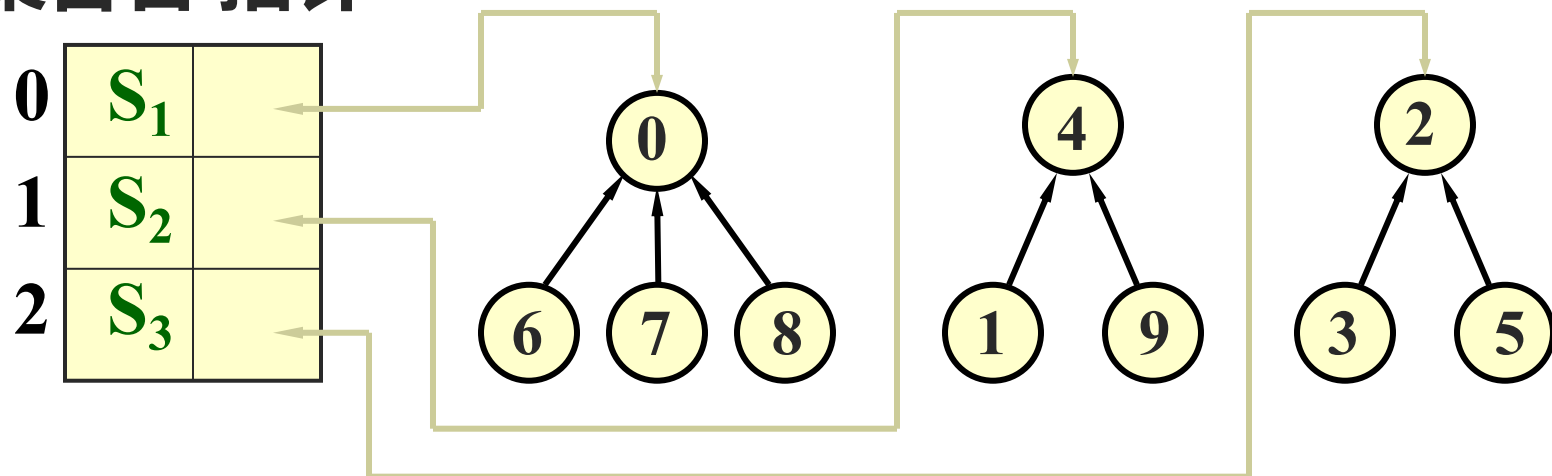


- 在双亲表示中，第 i 个数组元素下标代表包含集合元素 i 的树结点。根结点的双亲为 $-k$ ，表示集合中的元素个数为 k 。
- 同一棵树上所有结点所代表的集合元素在同一个子集合中。
- 为此，需要有两个映射：
 - a) 集合元素到存放该元素名的树结点间的对应；
 - b) 集合名到表示该集合的树的根结点间的对应。



- 设 $S_1 = \{0, 6, 7, 8\}$, $S_2 = \{1, 4, 9\}$, $S_3 = \{2, 3, 5\}$

集合名 指针

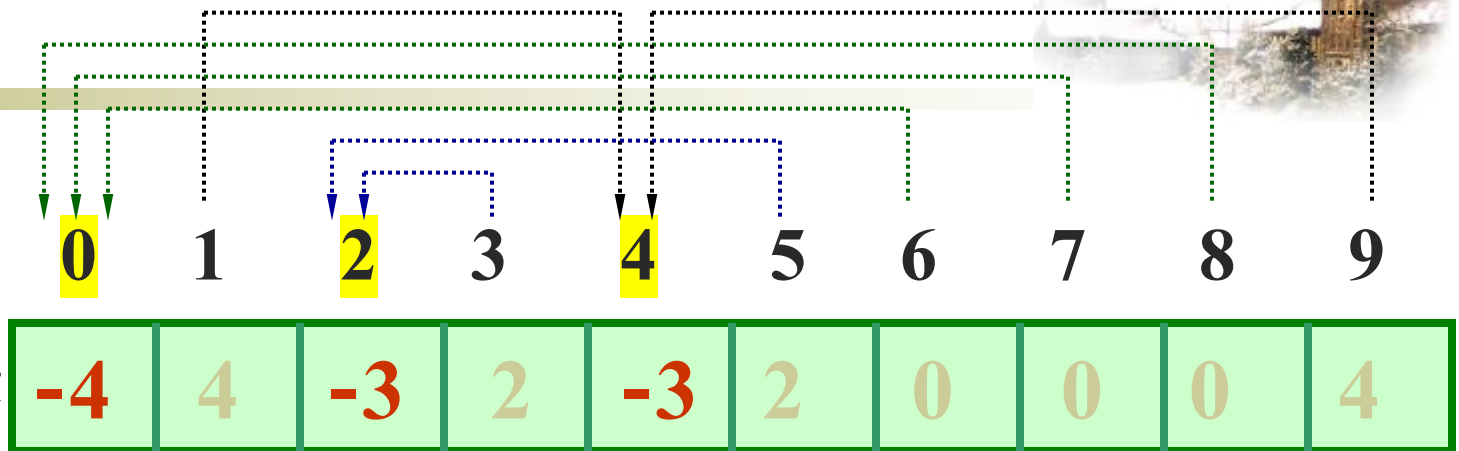


- 为简化讨论，忽略实际的集合名，仅用表示集合的树的根来标识集合。

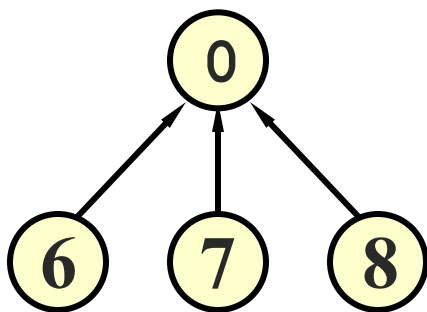


下标

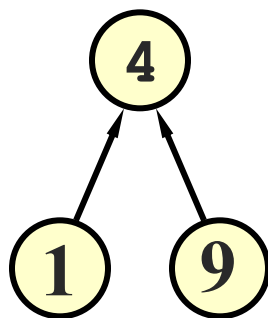
parent



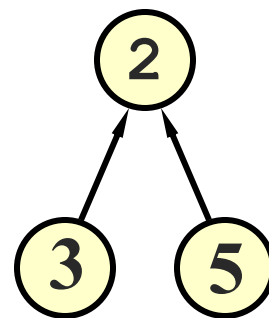
集合 S_1 , S_2 和 S_3 的双亲表示



S_1



S_2



S_3



下标

0

1

2

3

4

5

6

7

8

9

parent

-7

4

-3

2

0

2

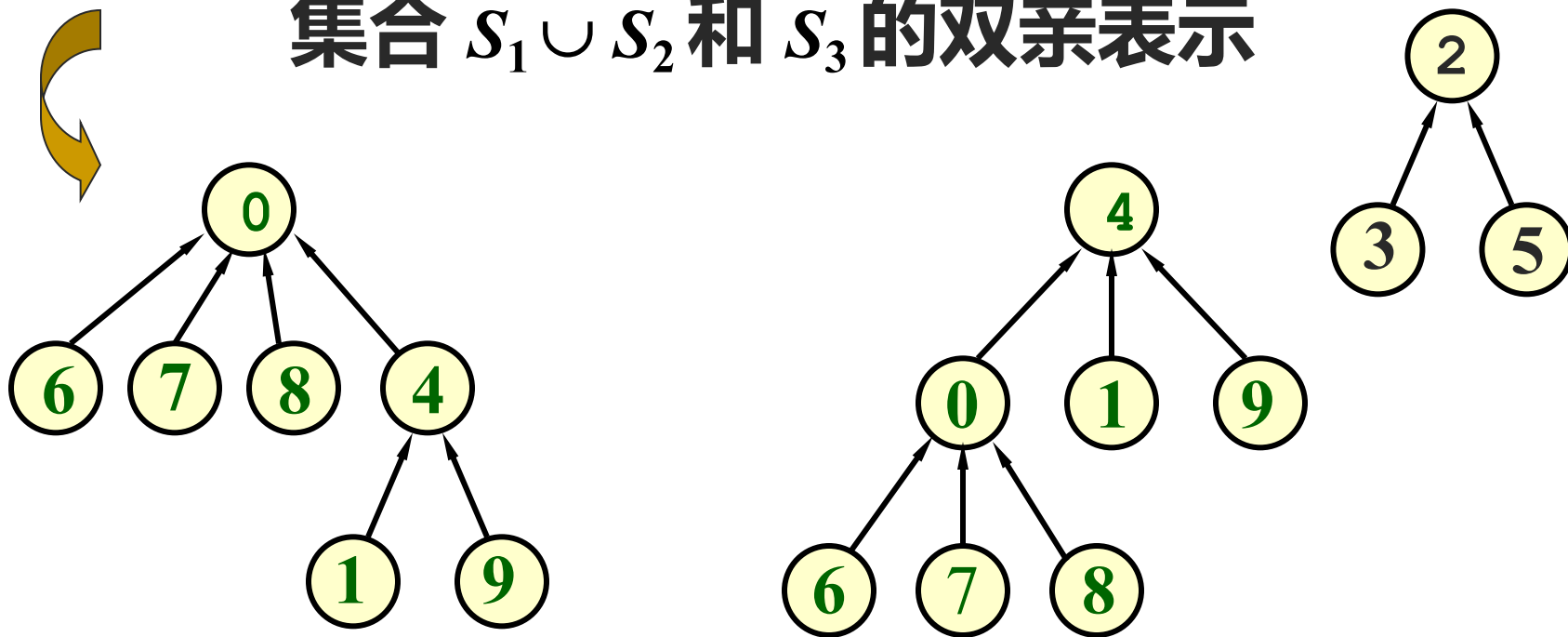
0

0

0

4

集合 $S_1 \cup S_2$ 和 S_3 的双亲表示



$S_1 \cup S_2$ 可能的表示方法



用双亲表示实现并查集的类型定义



```
const int DefaultSize = 10;
class UFSets {    //集合中的各个子集合互不相交
public:
    UFSets (int sz = DefaultSize);    //构造函数
    ~UFSets() { delete []parent; }    //析构函数
    UFSets& operator = (UFSets& R);    //集合赋值
    void Union (int Root1, int Root2);    //子集合并
    int Find (int x);    //查找x的根
    void WeightedUnion (int Root1, int Root2);
                                //改进例程：加权的合并算法
private:
```



```
int *parent;           //集合元素数组(双亲表示)
int size;              //集合元素的数目
};
```

```
UFSets::UFSets (int sz) {
//构造函数: sz 是集合元素个数, 双亲数组的范围
//为parent[0]~parent[size-1]。
```

```
    size = sz;           //集合元素个数
    parent = new int[size]; //创建双亲数组
    for (int i = 0; i < size; i++) parent[i] = -1;
                                //每个自成单元素集合
```

```
};
```



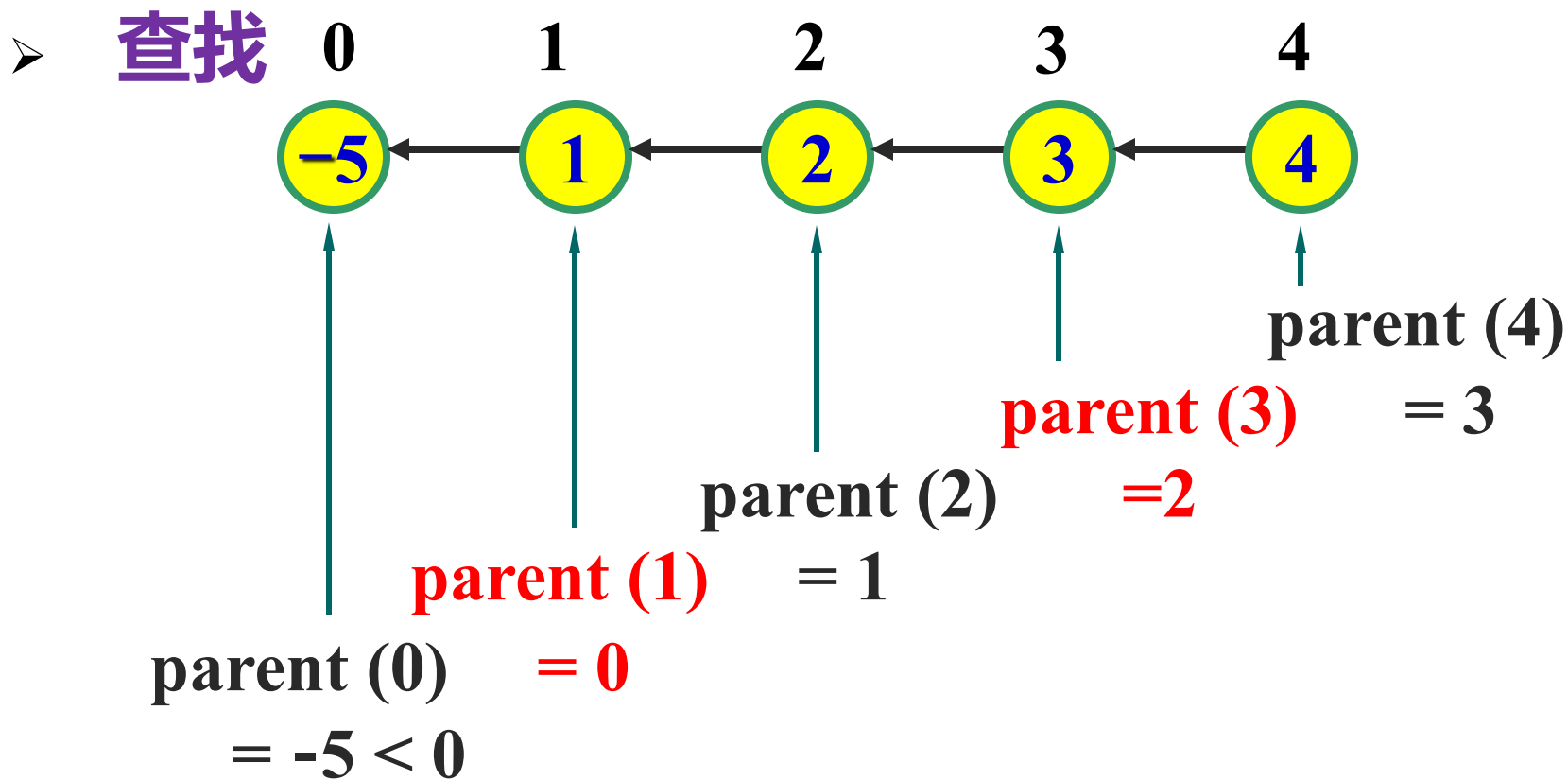
- 初始时, 用构造函数 $\text{UFSets}(s)$ 构造一个森林, 每棵树只有一个结点, 表示集合中各元素自成一个子集合。

下标	0	1	2	3	4	5	6	7	8	9
parent	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

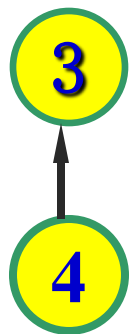
- 用 $\text{Find}(i)$ 寻找集合元素 i 的根。如果有两个集合元素 i 和 j , $\text{Find}(i) == \text{Find}(j)$, 表明这两个元素在同一个集合中;
- 如果两个集合元素 i 和 j 不在同一个集合中, 可用 $\text{Union}(i, j)$ 将它们合并到一个集合中。



• 并查集操作的算法



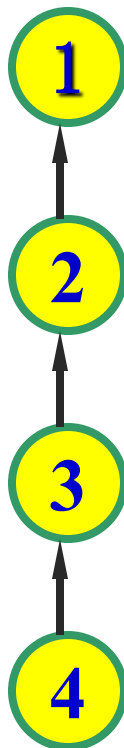
结束



Union(3,4)



Union(2,3)

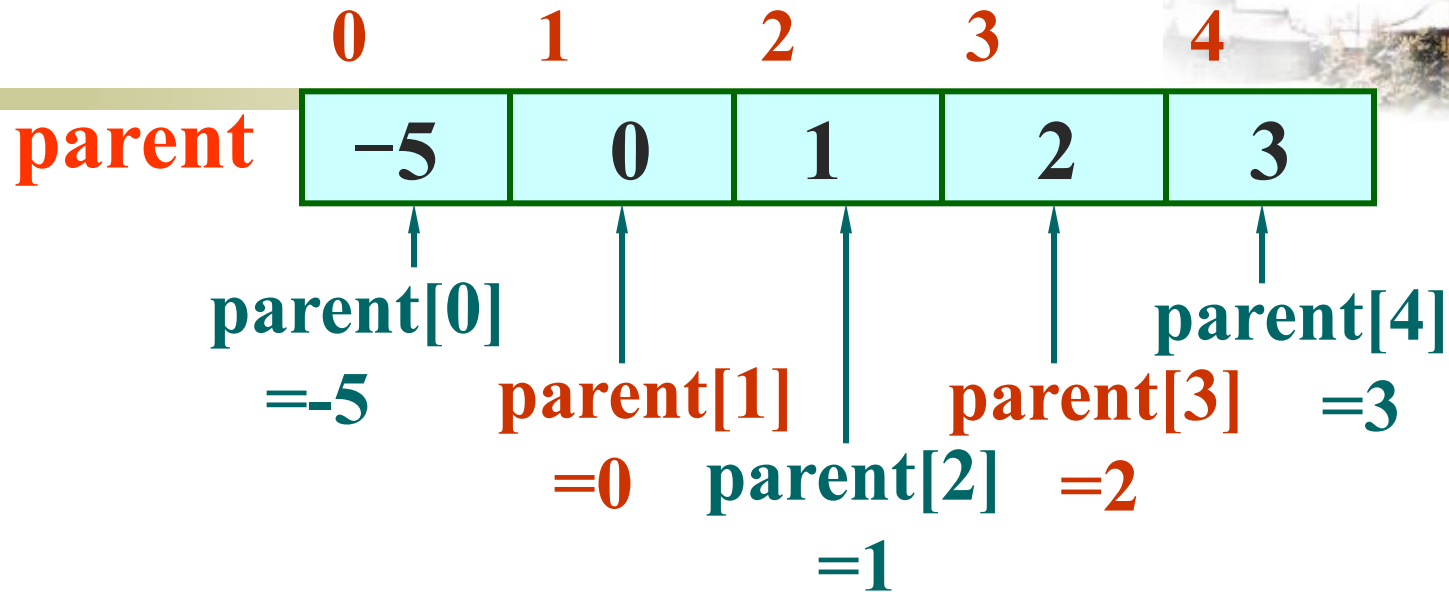


Union(1,2)



Union(0,1)

● **合并**



```
int UFSets::Find (int x) {  
    //函数搜索并返回包含元素x的树的根。  
    if (parent[x] < 0) return x; //根的parent[]值小于0  
    else return (Find (parent[x]));  
};
```



```
void UFSets::Union (int Root1, int Root2) {  
    //求两个不相交集合Root1与Root2的并  
    parent[Root1] += parent[Root2];  
    parent[Root2] = Root1;  
    //将Root2连接到Root1下面  
};
```

- Find 和 Union 操作性能不好。
- 假设最初 n 个元素构成 n 棵树组成的森林, $\text{parent}[i] = -1$ 。
- 做处理 $\text{Union}(n-2, n-1), \dots, \text{Union}(1, 2), \text{Union}(0, 1)$ 后, 将产生退化的树。



- 执行一次Union操作所需时间是 $O(1)$, $n-1$ 次Union操作所需时间是 $O(n)$ 。
- 若再执行Find(0), Find(1), ..., Find($n-1$), 若被搜索的元素为 i , 完成 Find(i) 操作需要时间为 $O(i+1)$, 完成 n 次搜索需要的总时间将达到

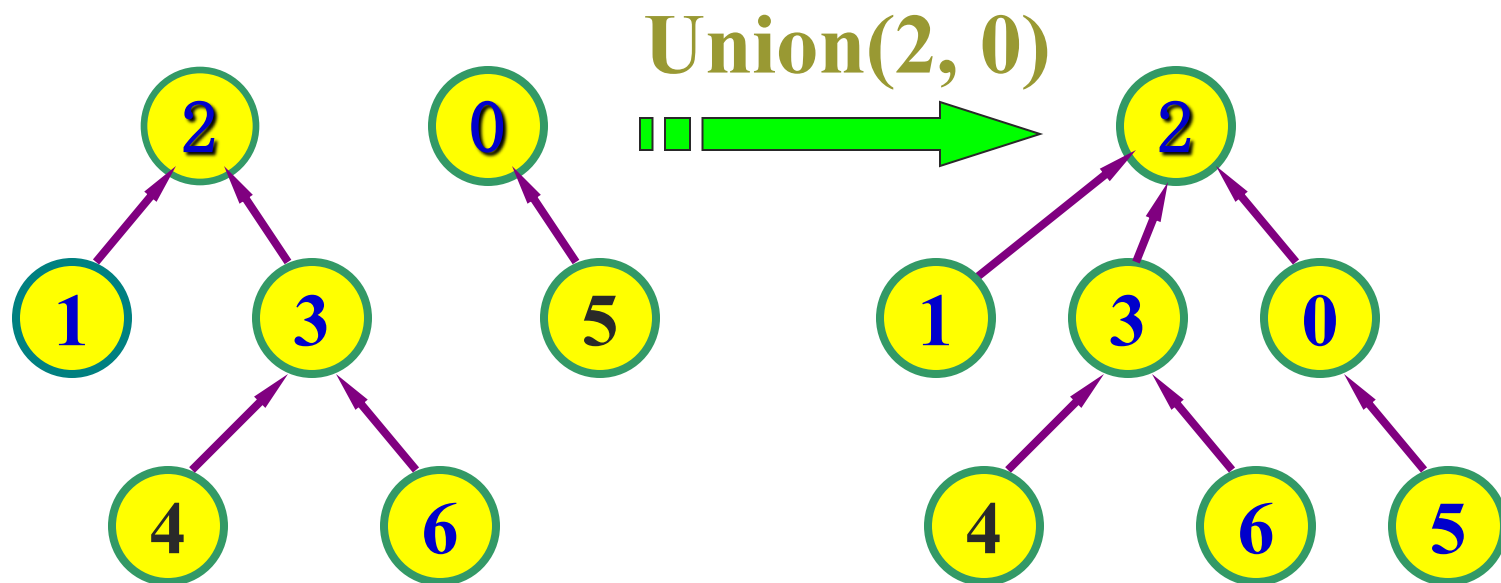
$$O\left(\sum_{i=0}^{n-1} (i+1)\right) = O(n^2)$$

- **改进的方法**
 - ◆ 按树的结点数合并
 - ◆ 按树的高度合并
 - ◆ 压缩元素的路径长度



- 按树结点个数合并(小树并入大树)

➤ 结点个数多的树的根结点作根





void UFSets ::

WeightedUnion (int Root1, int Root2) {

//按Union的加权规则改进的算法

int temp = parent[Root1] + parent[Root2];

if (parent[Root2] < parent[Root1]) {

parent[Root1] = Root2; //Root2中结点数多

parent[Root2] = temp; //Root1指向Root2

}

else {

parent[Root2] = Root1; //Root1中结点数多

parent[Root1] = temp; //Root2指向Root1

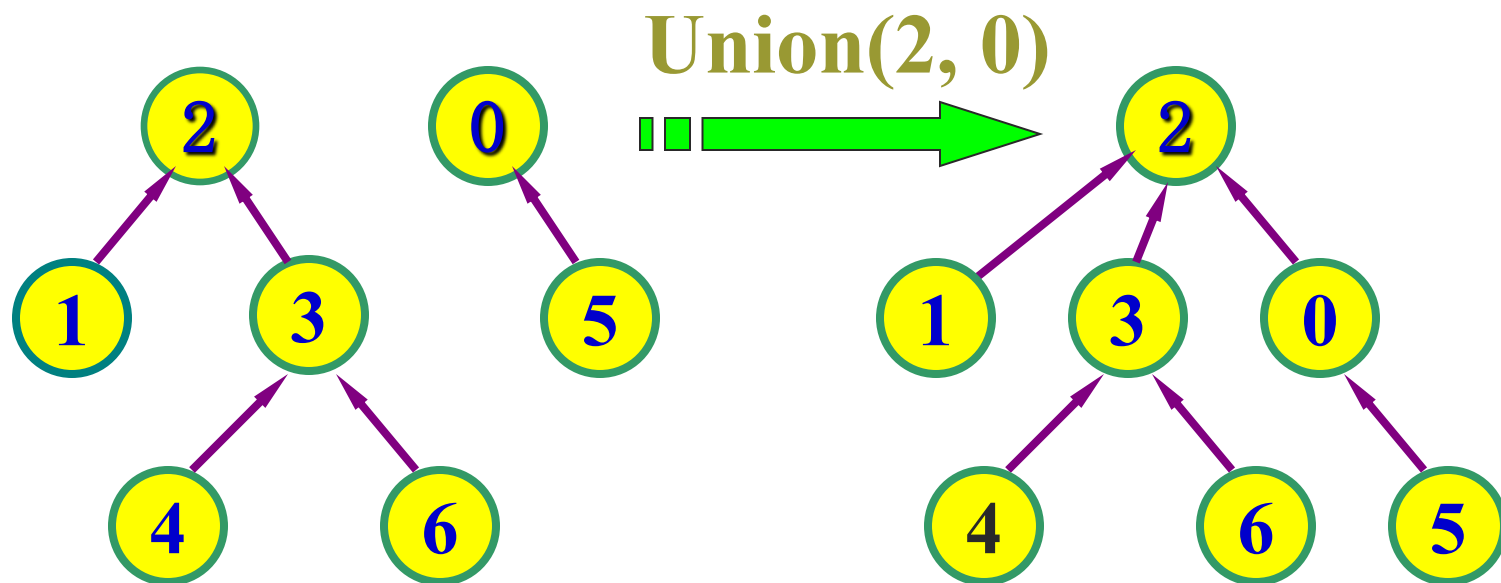
}

};



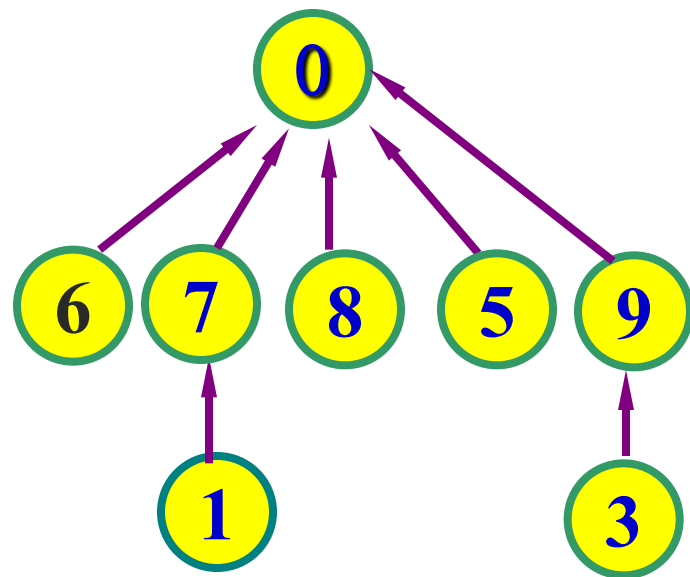
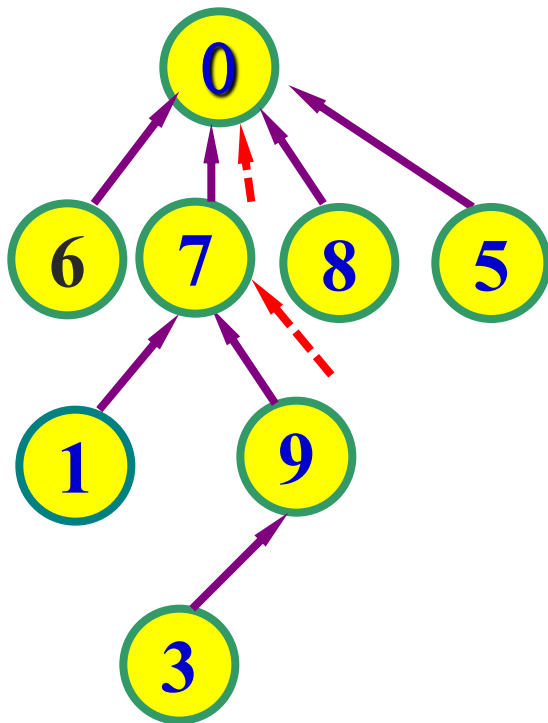
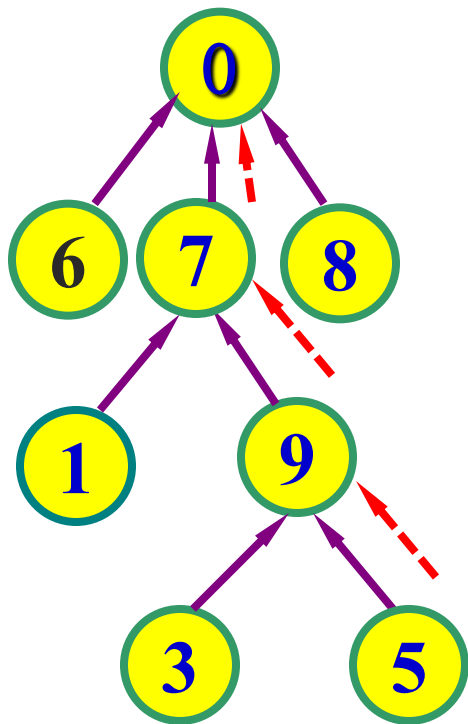
- 按树高度合并(矮树并入高树)

➤ 高度高的树的根结点作根





■ 压缩元素的路径长度(终极优化)





并查集

三要素

逻辑结构 —— 元素之间为“集合”关系

初始化 —— 初始化并查集，将所有数组元素初始化为 -1

Find ($S[]$, x) —— “查”，找到元素 x 所属集合

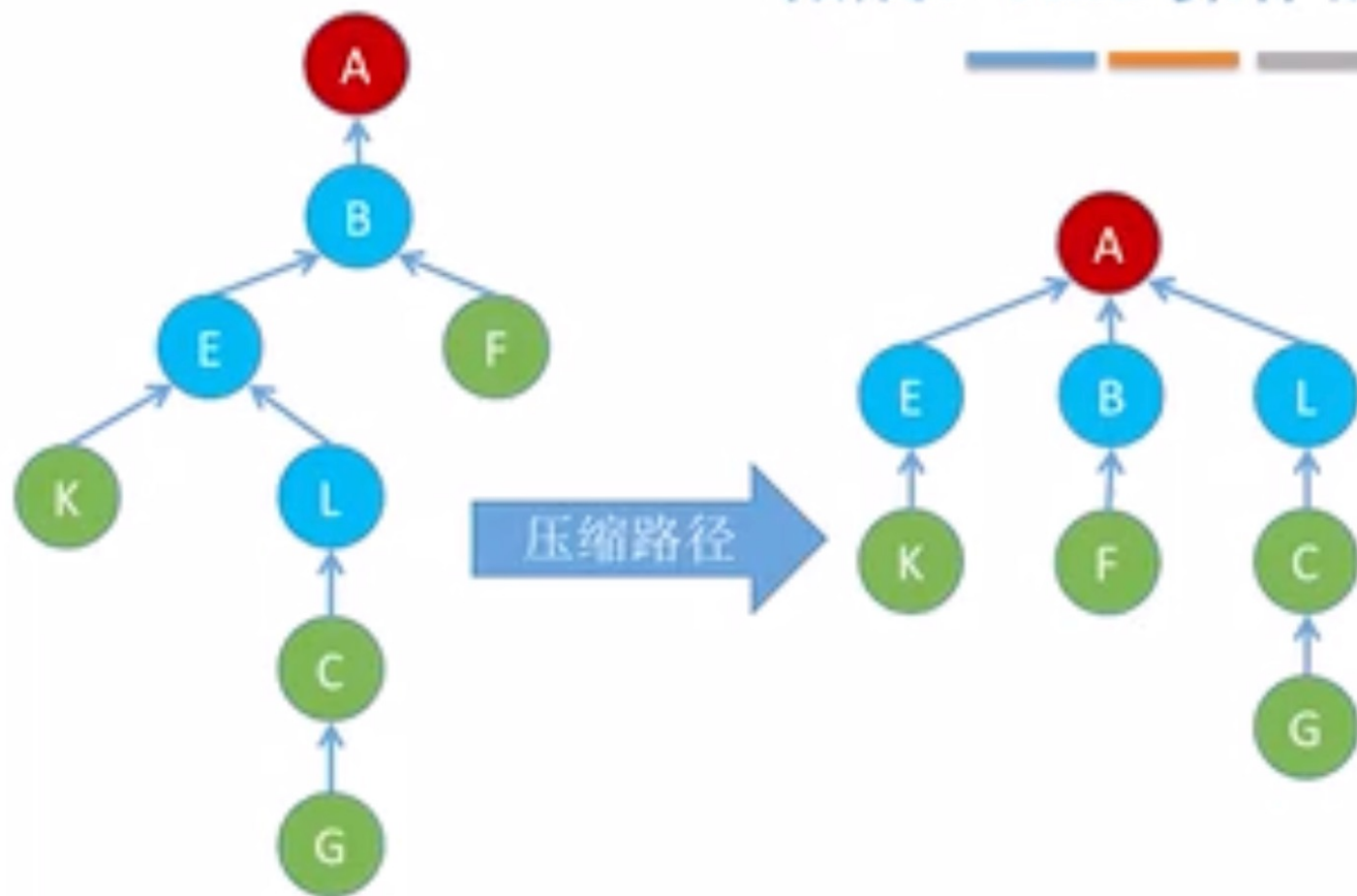
Union ($S[]$, $root1$, $root2$) —— “并”，将两个集合合并为一个集合

存储结构 —— 顺序存储，每一个集合组织成一棵树，采用“双亲表示法”

优化

用根节点的绝对值表示一棵树（集合）的结点总数

Union操作合并两棵树时，小树并入大树





用数组、双亲表示法来描述元素之间的集合关系。根节点为-1，非根节点指向父节点下标。

Find(int S[], x) —— 找到x所属集合
Union(int S[], int x, int y) —— 将x、y所属集合合并

Union
优化

用根节点的负值表示一棵树的结点总数

每次 Union 操作让小树合并到大树根节点下面

Find
优化

“压缩路径”的策略，每次 Find 操作先找到 x 所属根节点，再将查找路径上的所有结点都直接挂在根节点下面

最坏时间复杂度：

Find 操作 = 最坏树高 = $O(n)$

将n个独立元素通过多次Union合并为一个集合—— $O(n^2)$

最坏时间复杂度：

Find 操作=最坏树高= $O(\log_2 n)$

将n个独立元素通过多次Union合并为一个集合—— $O(n \log_2 n)$

最坏时间复杂度：

Find 操作=最坏树高= $O(\alpha(n))$

将n个独立元素通过多次Union合并为一个集合—— $O(n \alpha(n))$



- 在并查集数据结构中，通常使用一个数组 **parent[]** 来存储每个元素的 ____。初始化时，每个元素的父节点都是 ____，这表示每个元素自成一个集合。查找操作 **find(x)** 的目的是找到元素 **x** 所在集合的 ____。
- 在带路径压缩的 **find(x)** 操作中，递归查找根节点的过程中，会直接将查找路径上所有节点的父指针指向 ____。这种优化能将树的高度 ____，从而使得后续的查找操作时间复杂度接近 ____。



6.2.2 并查集的应用：等价类划分



等价关系与等价类(Equivalence Class)

- 在求解实际应用问题时常会遇到**等价类**问题。
- 从数学上看，等价类是对象（或成员）的集合，在此集合中所有对象应满足等价关系。
- 若用符号 “ $\equiv$ ” 表示集合上的**等价关系**，则对于该集合中的任意对象 x, y, z ，下列性质成立：
 - ◆ 自反性： $x \equiv x$ (即等于自身)。
 - ◆ 对称性： 若 $x \equiv y$ ，则 $y \equiv x$ 。
 - ◆ 传递性： 若 $x \equiv y$ 且 $y \equiv z$ ，则 $x \equiv z$ 。



- 因此，等价关系是集合上的一个**自反、对称、传递**的关系。
- “相等” (=) 就是一种等价关系，它满足上述的三个特性。
- 一个集合 S 中的所有对象可以通过等价关系划分为若干个互不相交的子集 $S_1, S_2, S_3, \dots$ ，它们的并就是 S 。这些子集即为**等价类**。



确定等价类的方法



确定等价类的方法分两步走：

1. 读入并存储所有的等价对 (i, j) ;
2. 标记并输出所有的等价类。



- 给定集合 $S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$, 及如下等价对: $0 \equiv 4, 3 \equiv 1, 6 \equiv 10, 8 \equiv 9, 7 \equiv 4, 6 \equiv 8, 3 \equiv 5, 2 \equiv 11, 11 \equiv 0$

- 进行归并的过程为:

初始 $\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}, \{9\}, \{10\}, \{11\}$

$0 \equiv 4$ $\{0, 4\}, \{1\}, \{2\}, \{3\}, \{5\}, \{6\}, \{7\}, \{8\}, \{9\}, \{10\}, \{11\}$

$3 \equiv 1$ $\{0, 4\}, \{1, 3\}, \{2\}, \{5\}, \{6\}, \{7\}, \{8\}, \{9\}, \{10\}, \{11\}$

$6 \equiv 10$ $\{0, 4\}, \{1, 3\}, \{2\}, \{5\}, \{6, 10\}, \{7\}, \{8\}, \{9\}, \{11\}$

$8 \equiv 9$ $\{0, 4\}, \{1, 3\}, \{2\}, \{5\}, \{6, 10\}, \{7\}, \{8, 9\}, \{11\}$

$7 \equiv 4$ $\{0, 4, 7\}, \{1, 3\}, \{2\}, \{5\}, \{6, 10\}, \{8, 9\}, \{11\}$



$\{0,4,7\}, \{1,3\}, \{2\}, \{5\}, \{6,10\}, \{8,9\}, \{11\}$

$6 \equiv 8 \quad \{0,4,7\}, \{1,3\}, \{2\}, \{5\}, \{6,8,9,10\}, \{11\}$

$3 \equiv 5 \quad \{0,4,7\}, \{1,3,5\}, \{2\}, \{6,8,9,10\}, \{11\}$

$2 \equiv 11 \quad \{0,4,7\}, \{1,3,5\}, \{2,11\}, \{6,8,9,10\}$

$11 \equiv 0 \quad \{0,2,4,7,11\}, \{1,3,5\}, \{6,8,9,10\}$



- 1. 在等价类划分问题中，并查集的**union**操作对应等价关系的什么性质？
 - A) 自反性
 - B) 对称性
 - C) 传递性
 - D) 反对称性
- 2. 处理完所有等价对后，并查集中具有相同根节点的元素属于：
 - A) 同一个等价类
 - B) 不同的等价类
 - C) 有序序列
 - D) 层次结构



- 分析以下场景是否适合用并查集解决：
 - "朋友关系"划分朋友圈
 - "师生关系"划分班级
 - "相等关系"划分等价类
 - "父子关系"构建家谱